

Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquín



Análisis de Ejercicio 2

Estudiantes:
Emily Lucila Menchú Menchú - 20472
Daniel Alfredo Nájera de Paz - 26961

Fecha: 17/08/2026

Análisis

Clases a definir:

- Main
- Torneo
- Atleta
- ControladorTorneo
- VistaTorneo

Relaciones entre clases:

- Main tendrá una relación de dependencia con ControladorTorneo porque solo la usará un momento.
- Torneo y Atleta tendrán una relación de agregación.
- ControladorTorneo y Torneo tendrán una relación de agregación.
- ControladorTorneo y Vista torneo tendrán una relación de agregación.

Preguntas:

¿Cómo determinará cuál es la siguiente posición disponible dentro del arreglo?

- Se recorrerá el arreglo tiempos usando un ciclo for hasta encontrar la primera posición cuyo valor sea 0, lo que indicaría que no ha sido usada. Además, antes de realizar un registro se usará el método getDisponibles() del atleta para verificar que todavía hay intentos disponibles.

¿Cómo recorrerá únicamente las posiciones del arreglo que contienen tiempos registrados?

- Se recorrerá el arreglo mediante un ciclo for que verificará las posiciones con valor mayor a 0, ya que 0 es el valor por defecto y nos indicará que no se ha registrado nada ahí. El método getIntentosRealizados() utiliza esta lógica para contar los intentos realizados del atleta.

Tabla de Propiedades

#	Tipo de Dato	Nombre	Visibilidad	Motivo	Clase
1	Atleta[]	atletas	Private	Almacenar los atletas disponibles del torneo.	Torneo
2	float[]	tiempos	Private	Almacenar los tiempos del atleta	Atleta
3	String	nombre	Private	Almacenar el nombre del atleta	Atleta
4	int	edad	Private	Almacenar la edad del atleta	Atleta
5	int	numeroParticipante	Private	Almacenar el número de participante del atleta	Atleta
6	int	numIntentos	Private	Número totales del atleta	Atleta

7	Torneo	torneo	Private	Guarda el torneo	ControladorTorneo
8	VistaTorneo	vista	Private	Guarda la vista	ControladorTorneo
9	Scanner	sc	Public	Guarda el scanner	VistaTorneo
10	int	error	private	Se usa para manejar la lógica de errores al ingresar datos que no son int	VistaTorneo

Tabla de métodos

#	Tipo de retorno	Nombre	Visibilidad	Parámetro	Motivo	Clase
1	-	Atleta	public	nombre:String, edad: int, numeroParticipante: int, numeroIntentos: int	Constructor de la clase Atleta.	Atleta
2	String	getNombre	public	-	Obtiene el nombre	Atleta
3	int	getEdad	public	-	Obtiene edad	Atleta
4	int	getNumeroParticipante	public	-	Obtiene número participante	Atleta
5	float[]	getTiempos	public	-	Obtiene los tiempos	Atleta
6	int	getIntentosRealizados	public	-	Retorna la cantidad de intentos realizados recorriendo la lista y buscando los mayores a 0	Atleta
7	int	getDisponibles	public	-	Obtiene disponibles calculando el total de intentos menos los realizados	Atleta
8	String	toString	public	-	Devuelve los datos del Atleta	Atleta
9		Torneo	public	Torneo(atletas: Atleta[])	Constructor de la clase torneo.	Torneo

10	Atleta	getAtleta	public	int:numeroParticipante	Obtiene el Atleta	Torneo
11	String	toString	public	-	Retorna String de los atletas	Torneo
12	-	ControladorTorneo	public	-	Constructor de ControladorTorneo	Controlador Torneo
13	void	mostrarMenuAtletas	public	-	Muestra el menú de atletas, valida valor ingresado y retorna el atleta escogido	Controlador Torneo
14	boolean	iniciarTomaTiempos	public	atleta: Atleta	Maneja el menú del atleta seleccionado y retorna un booleano para indicar si se debe finalizar el programa o regresar al menú de atletas.	Controlador Torneo
15	int	mostrarMenu	private	datosAtleta:String	Llama al menú de la vista, enviándole los datos del atleta y retorna la opción escogida	Controlador Torneo
16	void	registrarTiempo	private	tiempos: float[], disponible:boolean	Gestiona la pedida del tiempo, muestra mensajes de validación, valida el tiempo, la disponibilidad y registra el tiempo.	Controlador Torneo
17	void	modificarTiempo	private	tiempos:float[]	Gestiona la pedida del número de intento y del nuevo tiempo, hace validaciones, muestra mensajes y asigna el nuevo valor de tiempo.	Controlador Torneo
18	void	mostrarTiempo	private	tiempos:float[], todos:boolean	Gestiona la muestra del tiempo de cierto intento.	Controlador Torneo
19	void	calcularPromedio	private	tiempos:float[]	Realiza los cálculos de promedio	Controlador Torneo

20	void	mejorYMayorTiempo	private	tiempos:float[]	Realiza la lógica para determinar el mejor tiempo y mostrarlo	Controlador Torneo
21	void	mostrarIntentosDisponibles	private	atleta: Atleta	Gestiona la llamada de vista para mostrar intentos disponibles.	Controlador Torneo
22	-	VistaTorneo	public	-	Constructor de la vista. No tiene parámetros.	VistaTorneo
23	int	mostrarMenu	public	datosAtleta:String	Muestra el menú, junto a los datos del atleta y pide la opción al usuario para luego retornarla.	VistaTorneo
24	float	pedirTiempo	public	mensaje:String	Pide el tiempo al usuario	VistaTorneo
25	void	imprimirMensaje	public	mensaje:String	Imprime el mensaje que se el envíe como parámetro	VistaTorneo
26	int	pedirIntento	public	-	Pide el número de intento de tiempo a modificar	VistaTorneo
27	int	mostrarMenuAtleta	public	datosAtletas: String	Muestra los atletas disponibles	VistaTorneo
28	void	main	public	args: String[]	Punto de inicio del programa	Main